SMART CITY • CITIZEN SERVICES

CITY ISSUE REPORTING DASHBOARD



A MERN Stack Web Application

Report civic problems. Track progress. Improve city services.

Presented by: _____G . Naveen Kumar_____

Roll No(s): _____25MVCSMR0568_____

Department / College: _____ Academic Year: 2026–2027

OVERVIEW

Presentation Roadmap

From the civic problem to implementation, testing, and future scope.

1 **Problem & Need**

Why centralized reporting matters

2 **Objectives & Scope**

What the system delivers

3 **Features & Workflow**

Citizen and admin journeys

4 **MERN Architecture**

Frontend, API and database

5 **Implementation & Testing**

CRUD, security and validation

6 **Conclusion & Future**

Impact and possible extensions

Abstract

A single platform to capture, organize, and track city complaints.

The City Issue Reporting Dashboard is a web application built using MongoDB, Express.js, React.js, and Node.js. Citizens can submit civic issues with a description, category, location, and optional image. Administrators can review reports, manage records, and update issue status. MongoDB stores the data, while REST APIs connect the React frontend to the backend.

Project focus: full-stack development • CRUD • REST APIs • authentication • role-based access

C Citizen reporting

Submit structured reports with relevant details.

D Centralized records

Store and retrieve reports in MongoDB.

✓ Status visibility

Track reports from reported to resolved.

Problem Background

Common challenges when civic complaints are handled through disconnected channels.

Scattered reporting

Reports may arrive through calls, messages, or paper forms.

Limited tracking

Citizens may not know whether a report is pending or resolved.

Fragmented records

Manual records can be inconsistent and hard to search.

Slow prioritization

Categories and locations help identify recurring problems.

Core need: one place to report issues, store evidence, and monitor progress.

Problem Statement

Based on the supplied project brief.

Build a MERN-based city issue reporting dashboard that allows citizens to report city problems and supports management of issue category, location, image, and status using MongoDB. React provides the user interface, while Node.js and Express.js handle backend APIs. The system supports adding, viewing, updating, and managing records, with authentication and role-based access where required.

EXAMPLES OF CIVIC ISSUES

- Potholes
- Garbage
- Water leakage
- Streetlights
- Drainage

Project Objectives

Goals that guide design, implementation, and evaluation.

1 Demonstrate MERN

Integrate MongoDB, Express, React, and Node.js.

2 Implement CRUD

Create, read, update, and delete records.

3 Build REST APIs

Expose clear endpoints for issue management.

4 Capture issue details

Support category, location, image reference, and status.

5 Secure access

Use authentication and role checks where required.

6 Present results clearly

Show summaries, lists, filters, and status views.

Project Scope & Target Users

Separate first-version features from possible extensions.

IN SCOPE

- Registration and login
- Submit civic issue reports
- Store category, location, image reference and status
- View, update, and manage records
- Search and filter reports
- Admin dashboard and status management

FUTURE / OPTIONAL

- GPS and interactive map
- Automatic department assignment
- Email/SMS notifications
- Duplicate report detection
- Municipal system integration
- Mobile application and multilingual UI

Citizens submit and track reports • Administrators review and manage records.

Users & Responsibilities

Backend authorization must protect every sensitive operation.

U Citizen

- Register and sign in
- Submit a city issue
- View own reports
- Track report status

A Administrator

- View reports
- Edit issue records
- Update status
- Manage categories and summaries

M Maintenance staff (optional)

- View assigned issues
- Update progress
- Record resolution notes

Security rule: do not rely only on hiding buttons in React. Check identity, ownership, and role on the server.

Core Functional Features

Functional requirements derived from the project statement.

1 Issue form

Title, description, category, location, image

2 Dashboard

Summary cards and status counts

3 Issue list

Table/cards with detail view

4 Status workflow

Reported → In Progress → Resolved / Rejected

5 Search & filters

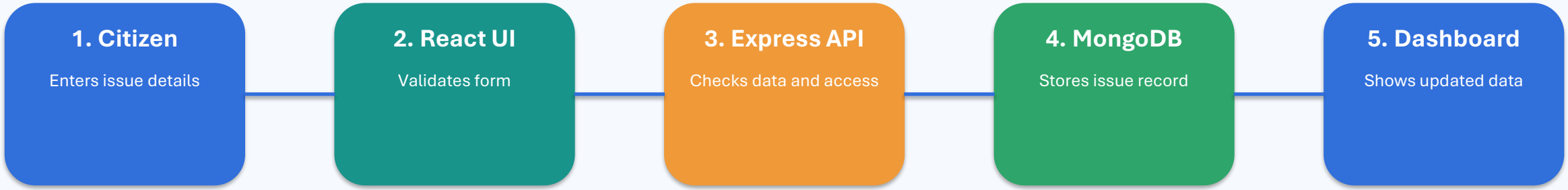
Find by category, location, and status

6 Record management

CRUD actions based on permissions

End-to-End System Workflow

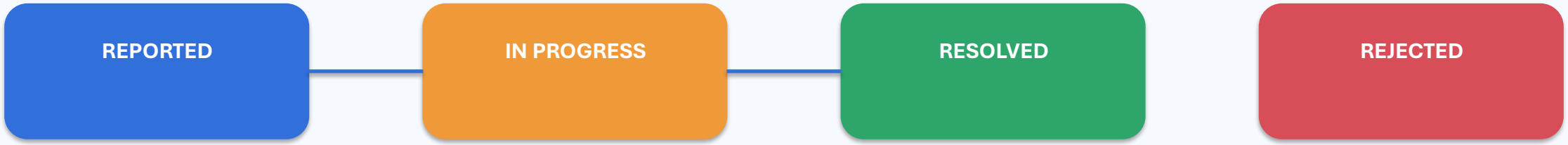
How a report moves from submission to status tracking.



Administrator reviews the report → updates status → backend saves changes → dashboard reflects latest status.

Issue Status Lifecycle

Use a small, consistent set of allowed status values.



Complaint received

Being investigated / fixed

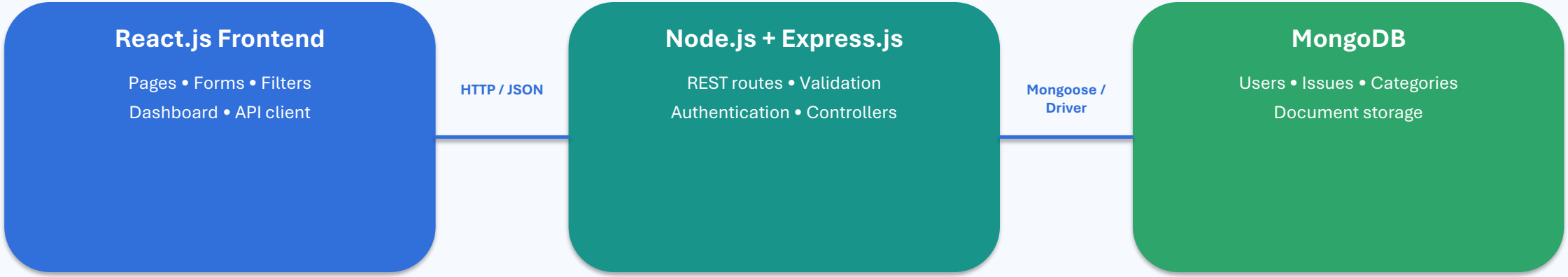
Work completed

Invalid / out of scope

Validate status transitions on the server. Timestamps and status history can improve accountability.

System Architecture

Three application layers communicate through REST APIs.

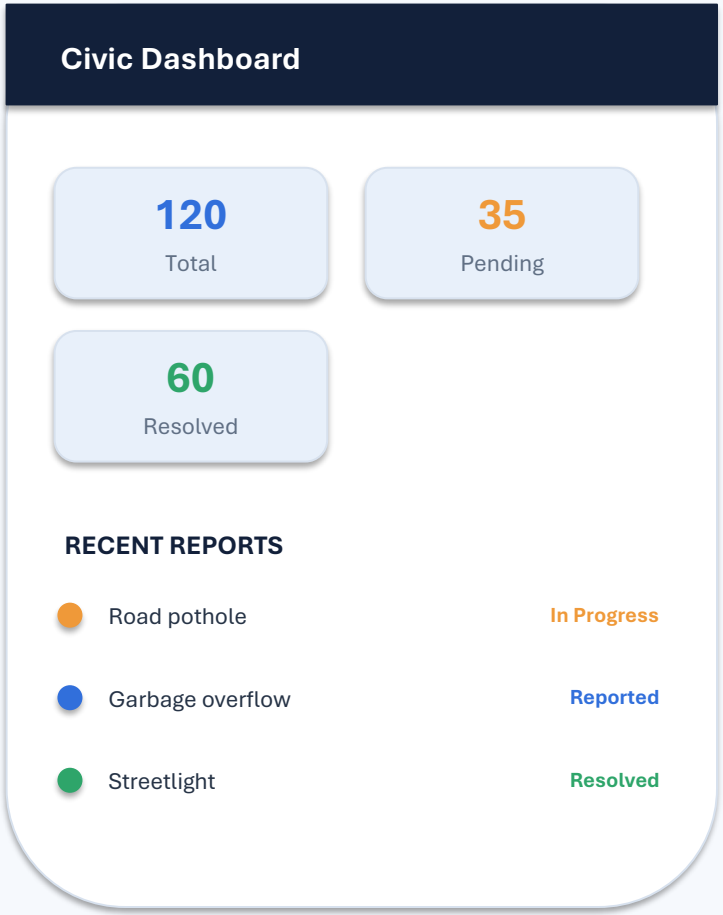


React request → Express middleware/controller → MongoDB operation → JSON response → UI update

Frontend: React.js

Reusable components make the interface easier to maintain.

- Login / Register: authentication forms and feedback
- Dashboard: summary cards and recent reports
- Report Issue: title, description, category, location, image
- Issue List / Details: search, filter, inspect and update permitted records
- Reusable components: navigation, issue cards, status badges, form controls



Illustrative UI mockup — not a live application screenshot.

Backend: Node.js & Express.js

The backend applies business rules and protects data.

1 Routes

Map HTTP methods and URLs to controller functions.

2 Controllers

Process requests and return consistent responses.

3 Validation

Check required fields, IDs and status values.

4 Authentication

Verify identity using a session or token.

5 Authorization

Enforce ownership and role permissions.

6 Error handling

Return appropriate HTTP codes and safe messages.

MongoDB Data Model

Recommended fields for the primary collections.

FIELD	EXAMPLE / TYPE
title	String: Damaged streetlight
description	String: Light not working
category	String or Category ObjectId
location	Address or GeoJSON
imageUrl	Image path or URL
status	Reported / In Progress / Resolved
reportedBy	User ObjectId
createdAt / updatedAt	Date timestamps

U

USERS collection

name • email • passwordHash • role

C

CATEGORIES (optional)

name • description • active flag

Passwords must be hashed. Store the image file in configured storage and save its URL/path in MongoDB.

CRUD Operations & REST APIs

Example endpoints; adapt them to the final code.

Operation	Method	Endpoint	Purpose
Create	POST	/api/issues	Create issue
Read all	GET	/api/issues	List authorized reports
Read one	GET	/api/issues/:id	View one report
Update	PUT	/api/issues/:id	Edit permitted fields
Update status	PATCH	/api/issues/:id/status	Change status
Delete	DELETE	/api/issues/:id	Delete if authorized
Statistics	GET	/api/dashboard/stats	Dashboard totals

Authentication endpoints may include POST /api/auth/register and POST /api/auth/login.

Algorithm: Submit a City Issue

Client and server validation help protect data quality.

01 Collect

Read issue details and optional image.

02 Validate

Check required fields and file constraints.

03 Authorize

Verify user identity and permissions.

04 Persist

Save issue data and image reference.

05 Respond

Return the created record or safe error.

06 Refresh

Update the list and show confirmation.

If validation fails, stop processing and return a clear validation message.

Authentication, Authorization & Validation

Security controls should be enforced on the server.

Password hashing

Store salted hashes, never plain-text passwords.

Role checks

Only authorized roles can perform admin actions.

Ownership checks

Users cannot modify other users' reports.

Input validation

Validate fields, IDs, allowed values and request sizes.

Upload protection

Limit file type/size and generate safe file names.

Secrets & errors

Use environment variables; do not expose secrets.

Do not rely only on hiding buttons in React. Protect every sensitive API route.

Development Tools & Setup

Typical tools used to build, test, and manage the project.

1 Visual Studio Code

Write and organize code.

2 Node.js + npm

Run server and manage packages.

3 MongoDB Atlas / Local

Database hosting and inspection.

4 Postman

Test REST endpoints and errors.

5 Git / GitHub

Version control and collaboration.

6 Browser DevTools

Debug UI and network calls.

Suggested structure: client/ for React; server/ for Express, models, routes, controllers and middleware.

Testing Plan & Test Cases

Test successful flows, invalid input, and unauthorized actions.

TEST CASE	ACTION / INPUT	EXPECTED RESULT
Registration	Submit valid details	Account created
Login	Wrong password	Access denied
Create issue	Valid report	Record saved
Required fields	Missing title/category	Validation error
Status update	Admin changes status	New status persists
Authorization	Citizen calls admin API	Request rejected
Image upload	Unsupported/oversized file	Upload rejected
Filter	Choose category/status	Matching reports shown

Performance & Quality Evaluation

Record actual results after running the application.

1 API response time

Measure common GET, POST and PATCH calls.

2 Dashboard load

Measure render and data-fetch time.

3 Database queries

Check filtering, sorting and pagination.

4 Error handling

Confirm meaningful status codes and messages.

5 Concurrent use

Try multiple users or repeated requests.

6 Usability

Check labels, contrast and mobile layout.

Do not invent response times, pass rates, or deployment claims. Add measurements from actual tests.

Expected Dashboard Output

Illustrative UI layout; replace with screenshots from the running project.

City Issue Dashboard

Search issues • Filter: All statuses

TOTAL ISSUES	REPORTED	IN PROGRESS	RESOLVED
120	35	25	60

RECENT REPORTS

Road pothole	Roads	Main Road	In Progress
Garbage overflow	Sanitation	Market Street	Reported
Water leakage	Water	Ward 4	Resolved

Sample values are illustrative and are not live database results.

Limitations & Assumptions

Important boundaries for a realistic first version.

- Report quality depends on accurate descriptions and locations entered by users.
- Image upload behavior depends on configured local or cloud storage.
- Official municipal integration is not included unless separately connected.
- Status accuracy depends on timely updates by authorized staff.
- Collect only necessary personal data and protect report access.

Future Enhancements

Potential extensions after the core dashboard is reliable.

1 Map & GPS

Show report locations with permission.

2 Notifications

Send email, SMS, or in-app updates.

3 Department assignment

Route reports to the right team.

4 Analytics

Identify recurring issues and hotspots.

5 Duplicate detection

Flag similar nearby reports.

6 Multilingual / mobile

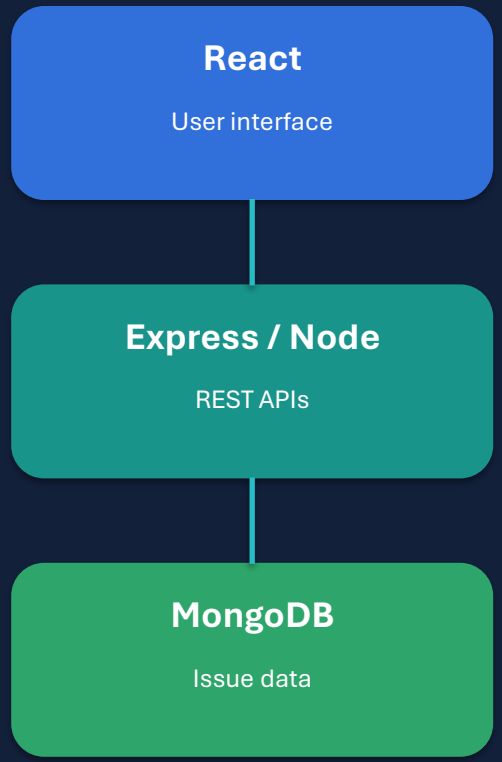
Support more languages and mobile access.

Conclusion

A practical full-stack approach to civic issue management.

The City Issue Reporting Dashboard provides a structured way to report and manage city problems. The MERN stack supports a responsive interface, REST-based communication, and persistent issue records.

- Centralized issue records
- CRUD through REST APIs
- Category, location, image reference and status tracking
- Authentication and role-based access where implemented
- Foundation for maps, notifications and analytics



References & Documentation

Official documentation for the technologies used.

- **MongoDB Documentation** https://www.mongodb.com/docs/
- **React Documentation** https://react.dev/
- **Node.js Documentation** https://nodejs.org/docs/
- **Express.js Documentation** https://expressjs.com/
- **Mongoose Documentation** https://mongoosejs.com/docs/
- **MDN Web Docs** https://developer.mozilla.org/
- **Postman Learning Center** https://learning.postman.com/

THANK YOU